Linear Regression - Revisited

Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State

Professor, Electrical and Computer (ECE) Engineering, NC State

NAIRR AI Education Fellow, Computing Research Association (CRA)

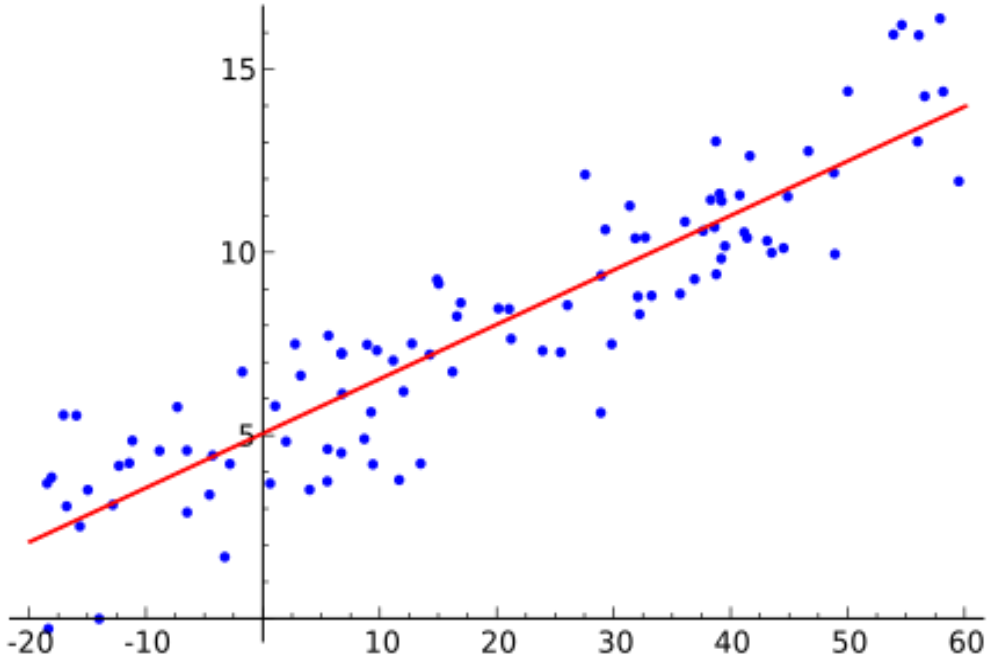
Linear Regression

- Given a training dataset $\mathcal{D}_{train} = \{(x_i, y_i)\}_{i=1}^{N}$ where $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$, we are looking for a function f_θ, such that

$$\hat{y}_i = f_\theta(x_i) = \theta_0 + \theta_1 \cdot x_{i,1} + \cdots + \theta_d \cdot x_{i,d}$$

- The parameters θ must be selected so this function fits the data as best as possible.

- If we $d = 1$, then we have $\hat{y}_i = \theta_0 + \theta_1 \cdot x_i$



Linear Regression

- To train this model, we need to fit it to our data. We do that by minimizing the error of our predicted values $\hat{y}$ when compared to the real / observed values y.

- We capture this error by using the so-called **loss function**. For regression, squared error is a common choice:

$$L\big(y_i, f_\theta(x_i)\big) = \big|\big|y_i - f_\theta(x_i)\big|\big|^2$$

- This **cost function** accumulates the prediction error over the entire training set:

$$C(\theta|\mathcal{D}_{train}) = \sum_{i=1}^{N} L\big(y_i, f_\theta(x_i)\big)$$

Linear Regression

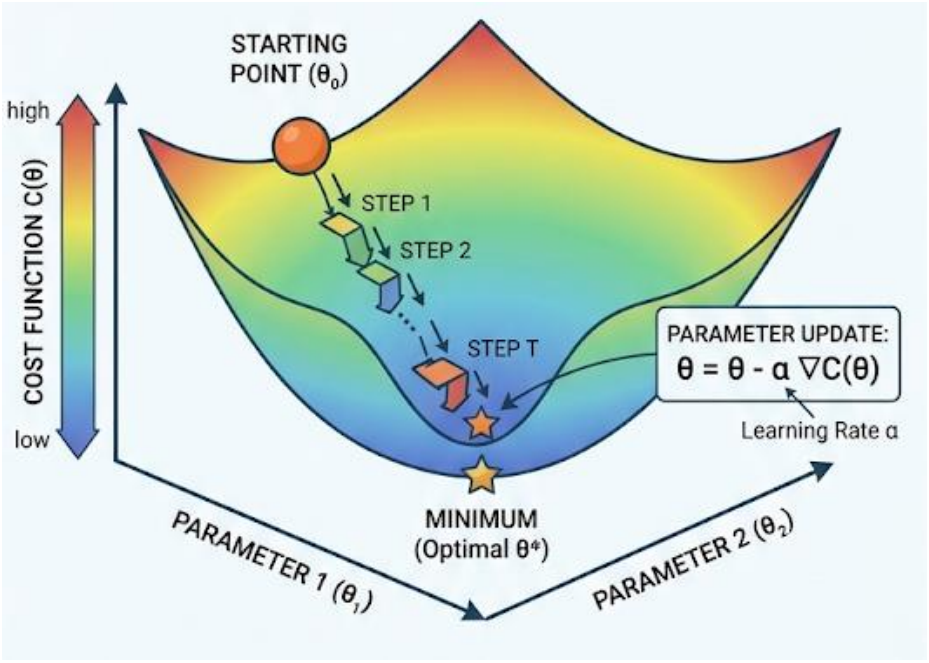
- We train a model by minimizing the cost function. That is,

$$\theta^* = \arg\min_{\theta} C(\theta|\mathcal{D}_{train})$$

- One way to solve for θ^* is to search for the critical point that satisfies

$$\frac{dC}{d\theta}(\theta^*) = 0$$

- Who remembers what **gradient descent** is?



Cost C(θ)
Defines the overall error to be minimized across the the landscape.

Gradient ∇C(θ)
Represents the slope and direction of steepest descent for the cost function.

Parameter Update
The mechanism for adjusting parameters to move doshhill.

Optimal Solution
The parameter set achieves the lowest possible cost.

Linear Regression

- A linear fit of a polynomial can be obtained by solving for the coefficients of each monomial in the model.

- If $f_\theta: \mathbb{R} \rightarrow \mathbb{R}$ then a linear fit of a degree d polynomial can be done by:

$$f(x_i) = \theta_0 + \theta_1 \cdot x_i + \theta_2 \cdot x_i^2 \ldots + \theta_d \cdot x_i^d$$

- How do you determine what degree is best for your data?
- The degree is the so-called **capacity** of the model. Why do you think it is called capacity?
- To choose a degree, we need to understand what we are trading-off (bias vs. variance)

How Computers Learn

- Consider the training data
  [1 1] → 2
  [2 1] → 3
  [2 2] → 4
- What is the output for an input [2 1]?
- What is the output for an input [3 2]?

- Consider the training
  [1 1] → 3
  [2 1] → 5
  [2 2] → 6
- What is the output for an input [3 2]?
  Relationship 1: $[x\ y] \rightarrow\ 2x + y = 8$
  Relationship 2: $[x\ y] \rightarrow\ x^2 - x + y + 2 = 10$

We test an ML model by giving it data that it has not seen to ensure that it is capturing the underlying relationship. [**Generalization**]

The model needs to be properly specified given the amount of data provided. [**Overfitting / Underfitting**]

Bias vs. Variance

Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State

Professor, Electrical and Computer (ECE) Engineering, NC State

NAIRR AI Education Fellow, Computing Research Association (CRA)

Bias: When the Model Is Too Simple

Underfitting — systematic error from assumptions that don't match reality

What It Is

Error from oversimplified assumptions — the model can't capture the true pattern in the data.

Symptoms

High error on both training and test sets; predictions barely shift with new data.

Common Causes

Too few parameters, an overly rigid model family, or excessive regularization.

How to Reduce It

Add model capacity, engineer richer features, or ease up on regularization.

High-bias models underfit — they're consistently wrong in the same direction.

Variance: When the Model Is Too Sensitive

Overfitting — the model reacts to noise instead of signal

What It Is

Error from sensitivity to small fluctuations in the training set — the model fits noise, not signal.

Symptoms

Low training error but high test-error; predictions swing sharply across different training samples.

Common Causes

Too many parameters, too little training data, or too little regularization.

How to Reduce It

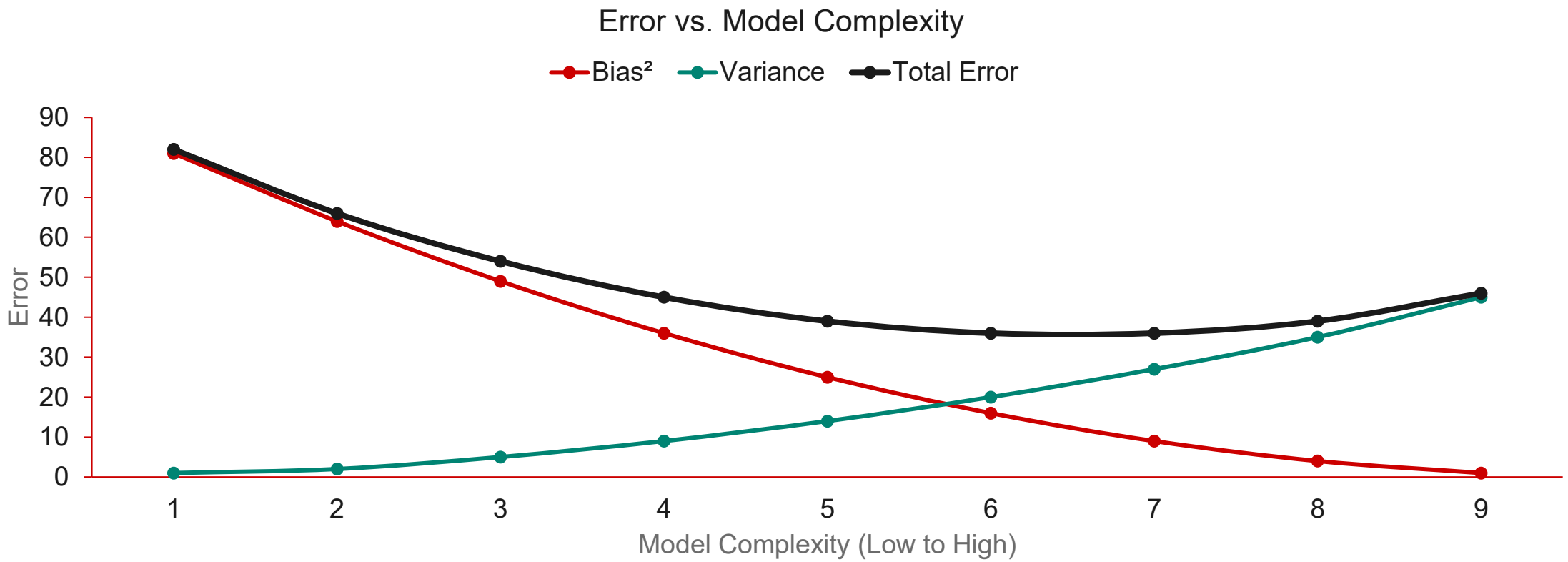
Collect more data, add regularization, simplify the model, or use ensembling.

High-variance models overfit — they change dramatically with the training data.

The Bias-Variance Trade-off

NC STATE Engineering

Total error is minimized where the two error sources balance



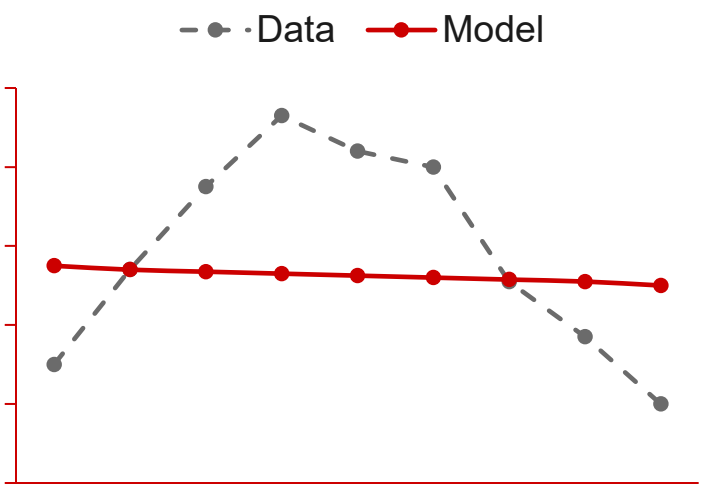
The best model isn't the most complex one — it's the one that minimizes total error.

What Underfitting and Overfitting Look Like

NC STATE Engineering

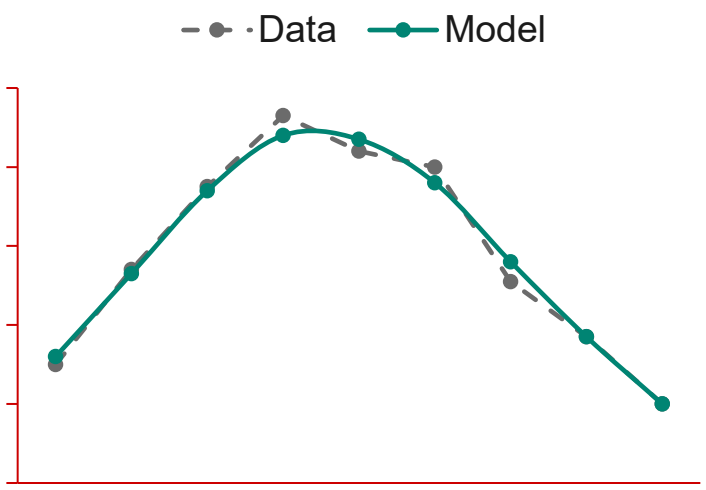
Same underlying data, three very different model fits

Underfit



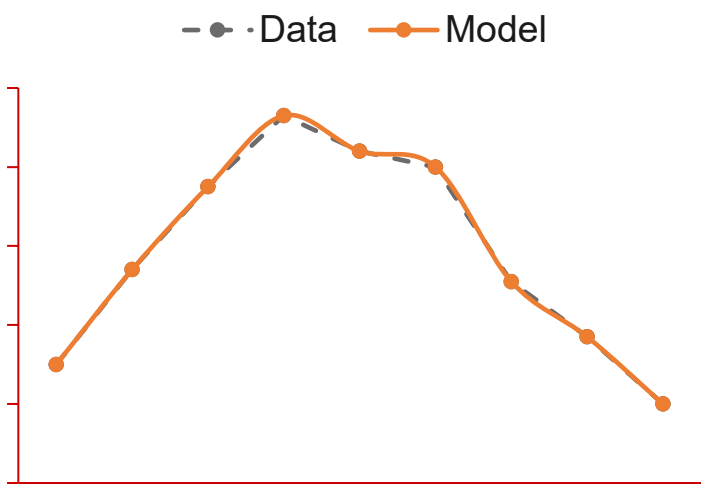
The model ignores the pattern — too rigid to bend toward the data.

Good Fit



The model tracks the true trend without chasing every fluctuation.

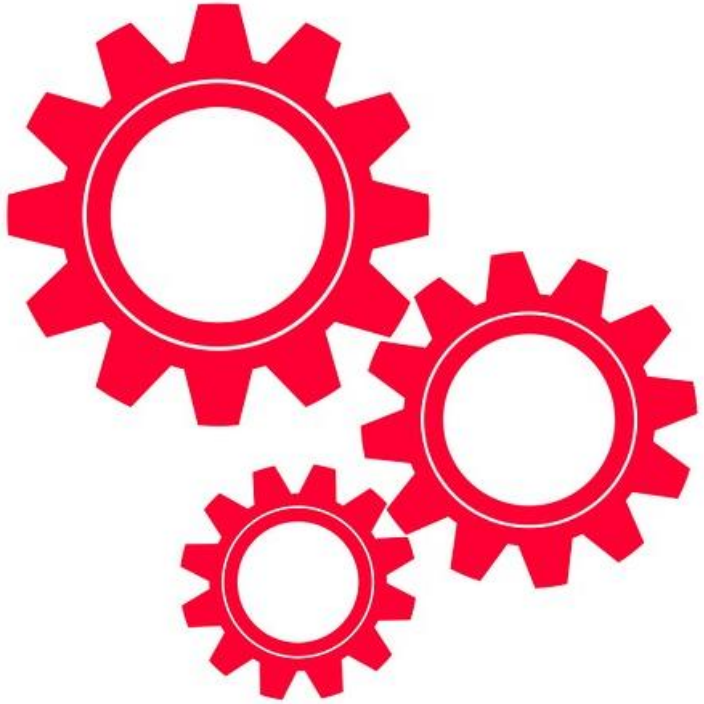
Overfit



The model memorizes the noise — it won't generalize to new data.

The goal isn't a perfect fit to training data — it's a model that generalizes.

Bias vs. Variance Tradeoff



Multi-Label Logistic Regression

Edgar Lobaton, Ph.D.

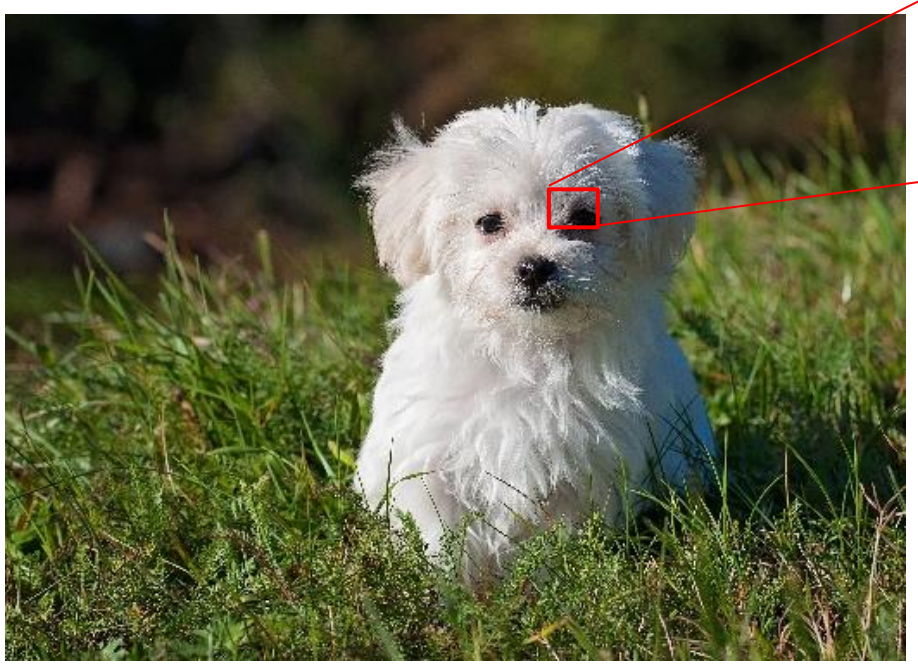
Director of Applied AI, College of Engineering (COE), NC State

Professor, Electrical and Computer (ECE) Engineering, NC State

NAIRR AI Education Fellow, Computing Research Association (CRA)

Understanding the Input and Output

Images are arrays of pixels value. All that the computer “sees” is a list of those numbers.



The labels are often encoded using a list as well. For example, if our list contains cats, dogs and birds, we could use a one-hot encoding:

$$Cat \rightarrow [1\ 0\ 0]\ ;\ Dog \rightarrow [0\ 1\ 0]\ ;\ Bird \rightarrow [0\ 0\ 1]$$

Logistic Regression

- You may recall that for a binary problem, where the output is either 0 or 1, the prediction $\hat{y}$ of a model is associated with the probability of having a positive response (i.e., 1). Mathematically,

$$\hat{y}_i = f_\theta(x_i) = P[y_i = 1 | x_i]$$

- For logistic regression, we had:

$$\hat{y}_i = \sigma(\theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d)$$

where σ is the **sigmoid / logistic function**:

$$\sigma(s) = \frac{1}{1 + e^{-s}} \in [0,1]$$

Soft-max Regression

- When we have multiple labels, we represent y as the vector associated with the likelihood of getting a specific label, e.g. $y_i = [y_{i,1}, y_{i,2}, y_{i,3}]^\top \in \mathbb{R}^D$ in our cat/dog/bird example indicates that $y_{i,1}$ is the probability of having a cat and so on.

- In this case, we use

$$\hat{y}_i = f_\theta(x_i) = \sigma(Wx_i + b)$$

where $W \in \mathbb{R}^{D \times d}$, $b \in \mathbb{R}^D$ and σ is the **soft-max function**

$$\sigma_k(s) = \frac{e^{s_k}}{\sum_j e^{s_j}} \in [0,1]$$

- Note that $\sum_k y_{i,k} = 1$. We call $a_i = Wx_i + b$ the **logits**. These are unnormalized values.

Soft-max Regression

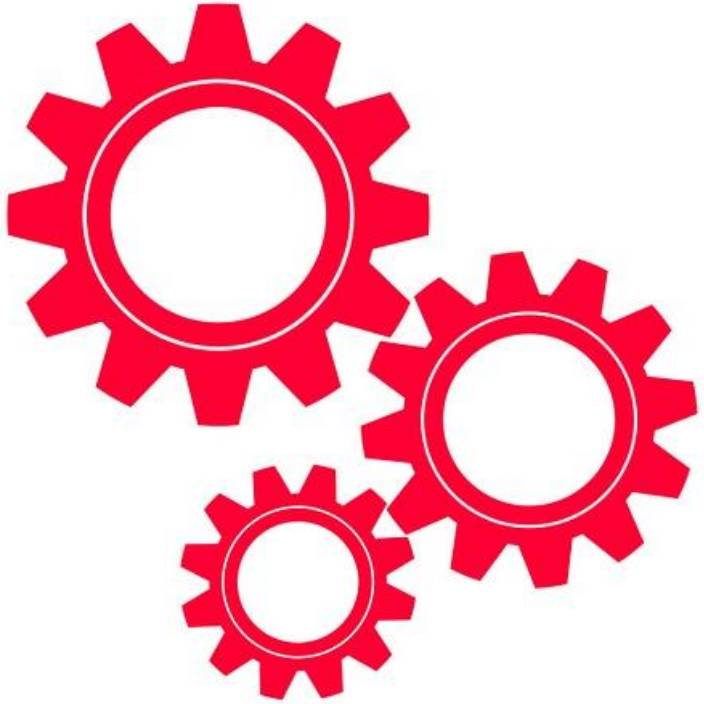
- For classification, the **cross-entropy loss** is a common choice:

$$L(y_i, \hat{y}_i) = \sum_k y_{i,k} \cdot \ln(\hat{y}_{i,k})$$

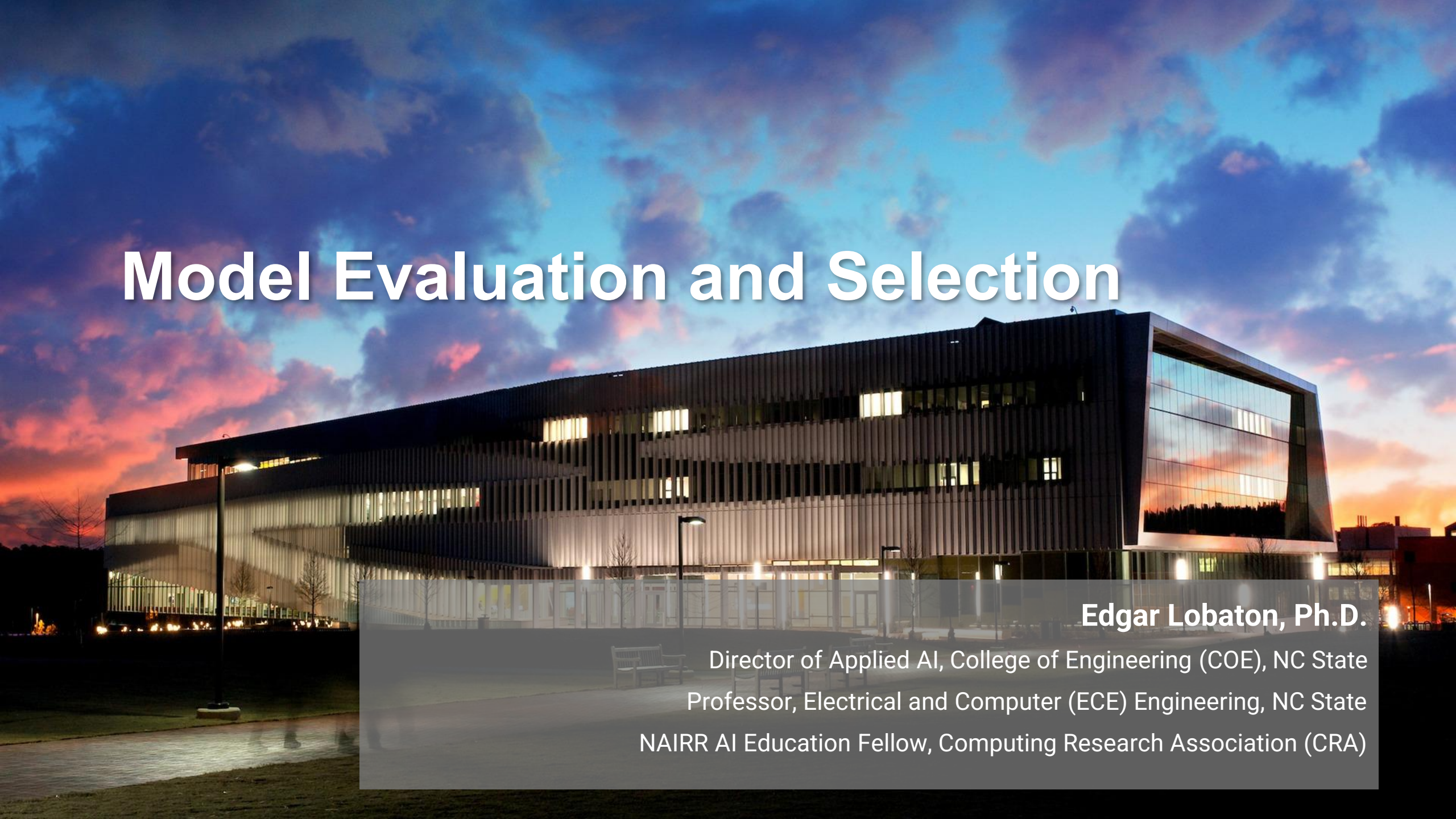
- The cost function for the entire training set would be:

$$C(\theta|\mathcal{D}_{train}) = \sum_{i=1}^{N} L(y_i, \hat{y}_i)$$

Logistic Regression



Model Evaluation and Selection



Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State

Professor, Electrical and Computer (ECE) Engineering, NC State

NAIRR AI Education Fellow, Computing Research Association (CRA)

From Training Error to Generalization

Training accuracy alone can't tell us how a model will perform on new data

The Problem

A model can reach near-zero training error yet fail completely on data it has never seen.

Why It Happens

High-variance models memorize specific training examples instead of learning general patterns.

The Fix

Hold out data the model never trains on and use it to estimate real-world performance.

The Goal

Choose the model and settings that generalize best — not the one that memorizes best.

Evaluation is how we find out if a model actually learned — or just memorized.

The Train / Validation / Test Split

NC STATE Engineering

One dataset, three separate jobs — each set is used differently

Fit the Model

Used to directly learn the model's parameters through training.

Tune & Select

Compares models and hyperparameters to pick the best configuration.

Final Check

Touched once, at the end, for an unbiased performance estimate.

Exact ratios vary with dataset size — but the roles of each split never change.

Why We Need a Validation Set

A held-out set for making decisions — without ever touching the test set

Model Selection

Compare different model types or architectures fairly, using data they never trained on.

Hyperparameter Tuning

Choose settings like learning rate, tree depth, or regularization strength.

Early Stopping

Detect the point where a model starts to overfit during training and stop there.

Protecting the Test Set

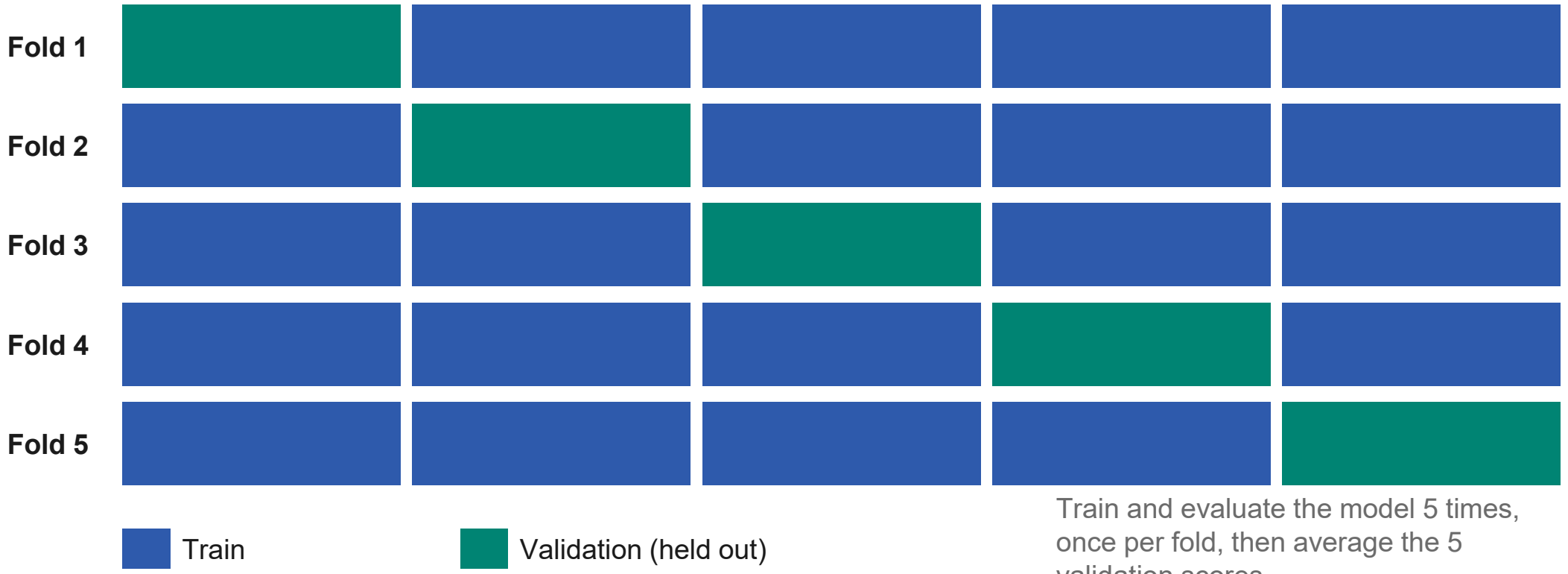
Keep the test set untouched so its final performance estimate stays unbiased.

The validation set lets you experiment freely — the test set stays locked until the very end.

Cross-Validation: Stretching Limited Data

NC STATE Engineering

Rotate the validation fold so every example is used for both training and validation



Train and evaluate the model 5 times, once per fold, then average the 5 validation scores.

Cross-validation trades extra computation for a more reliable performance estimate.

Putting It Together: The Model Selection Workflow

NC STATE Engineering

The test set is touched exactly once — at the very end

Divide examples into training, validation, and test sets.

Fit several models or hyperparameter settings on the training set.

Score every candidate on the validation set.

Pick the configuration with the strongest validation performance.

Report performance on the test set — a single, unbiased look.

Only step 5 touches the test set — and only once.

From Classical ML to Deep Learning

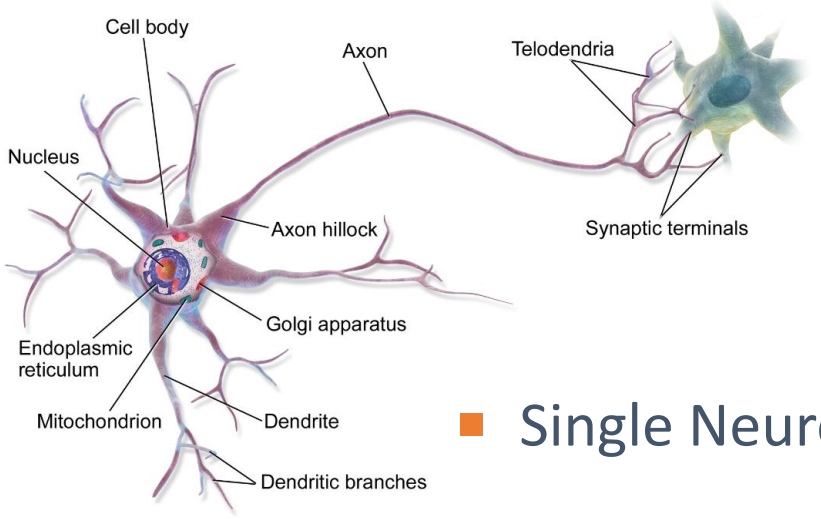
Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State

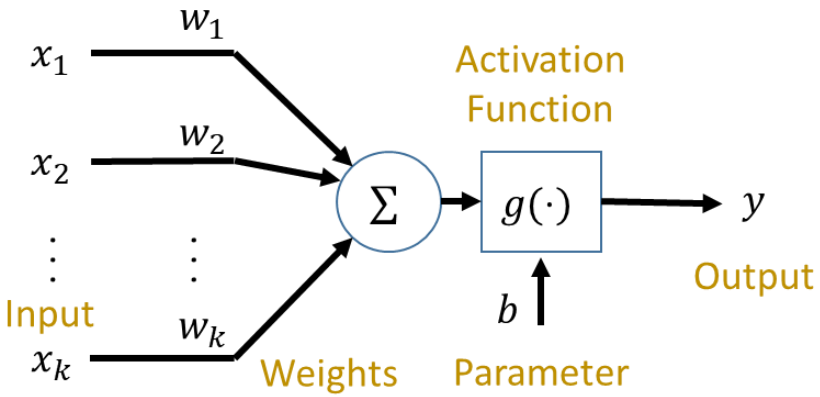
Professor, Electrical and Computer (ECE) Engineering, NC State

NAIRR AI Education Fellow, Computing Research Association (CRA)

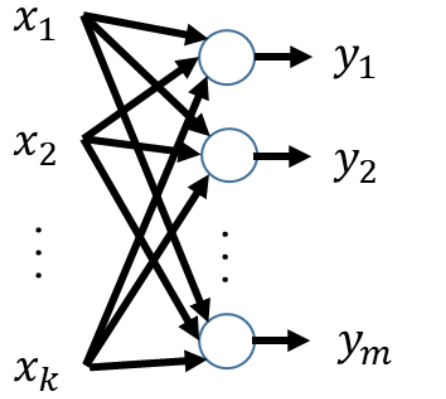
Neural Nets



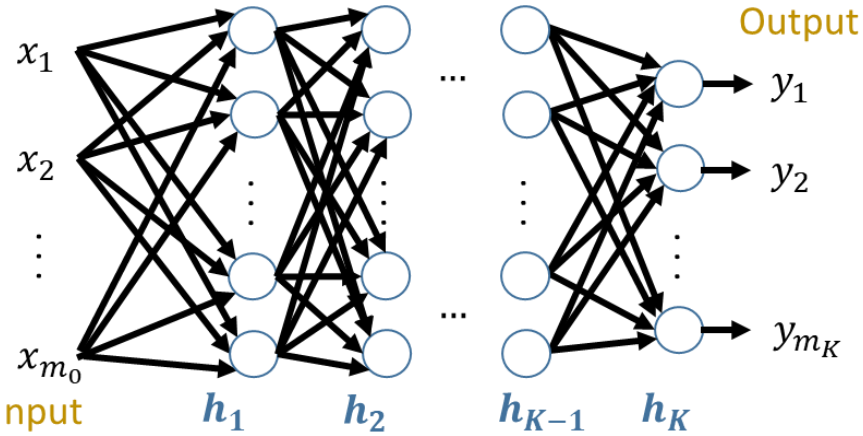
- Single Neuron



1-layer NN



K-layer NN



* Neurons figure modified from https://en.Wikipedia.org

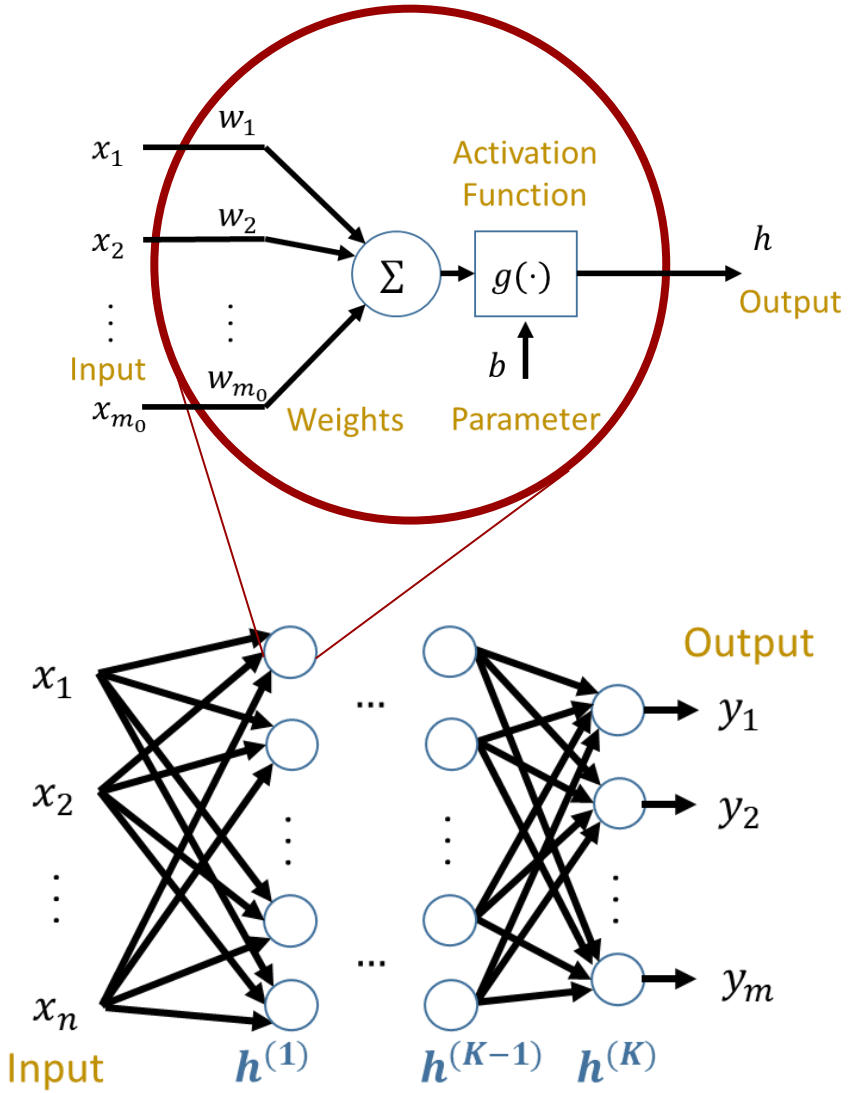
MLP Architecture

- Remember that for a single neuron we have

$$h = g(Wx + b)$$

where $W = [w_1, w_2, \cdots, w_{m_0}] \in \mathbb{R}^{1 \times m_0}, b \in \mathbb{R}$ and $x \in \mathbb{R}^{m_0}$

- A **Multi-Layer Perceptron (MLP)** is formed by stacking neurons in layers and concatenating layers into a network.



MLP Architecture

- When stacking neurons into a layer, we end up with a similar expression

$$h = g(Wx + b)$$

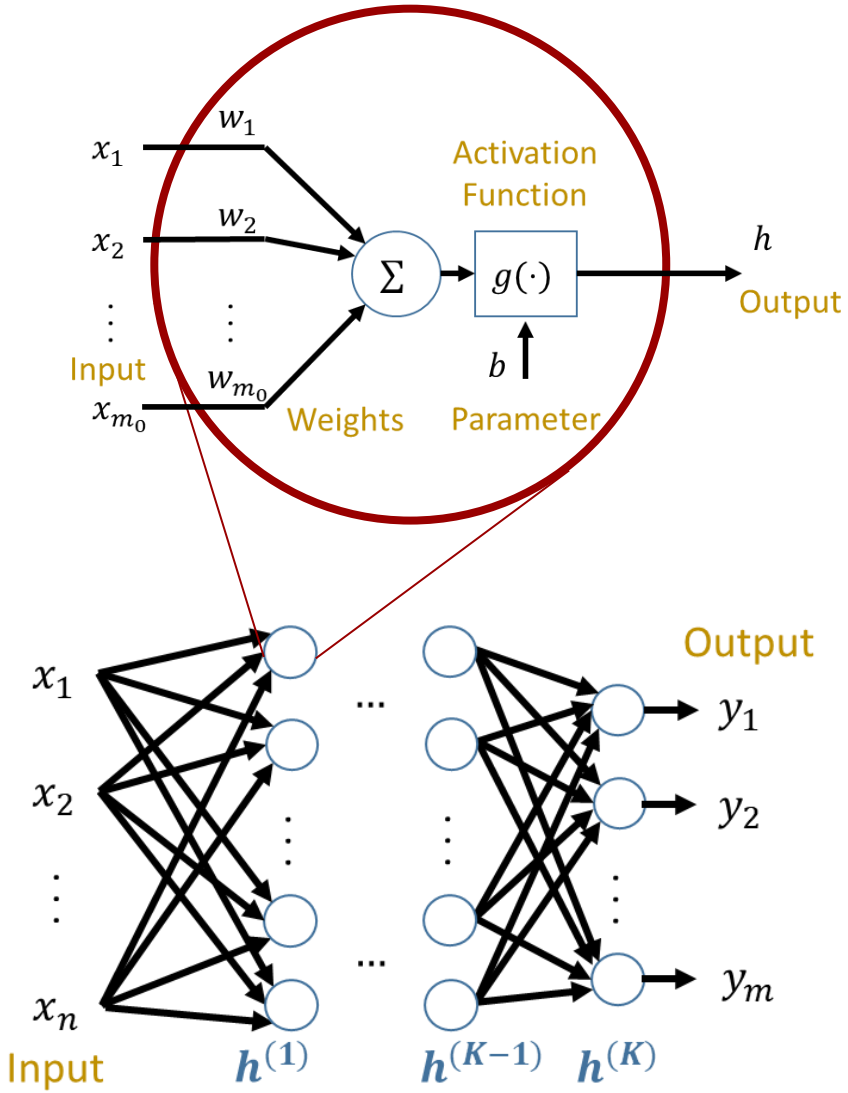
where $W \in \mathbb{R}^{m_1 \times m_0}, b \in \mathbb{R}^{m_1}$ and $x \in \mathbb{R}^{m_0}$

- For K layers, we end up with:

$$h^{(0)} = x \in \mathbb{R}^{m_0}$$

$$a^{(k)} = W^{(k)} h^{(k-1)} + b^{(k)}$$
$$h^{(k)} = g^{(k)}\left(a^{(k)}\right)$$

$$\hat{y} = h^{(K)}$$



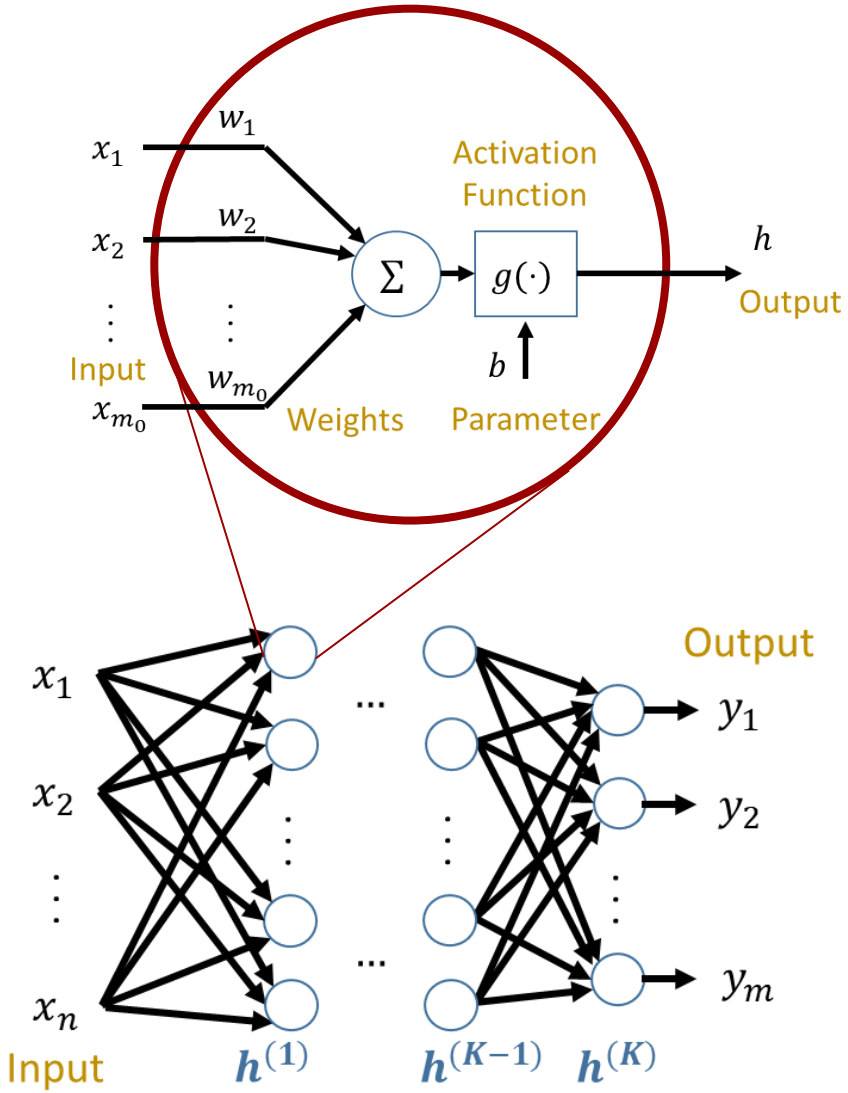
MLP Architecture

- For a single layer case, $K = 1$, we get

$$h^{(0)} = x$$
$$h^{(1)} = g\left(Wh^{(0)} + b\right)$$
$$\hat{y} = h^{(1)}$$

- Combining everything, we get $\hat{y} = g(Wx + b)$

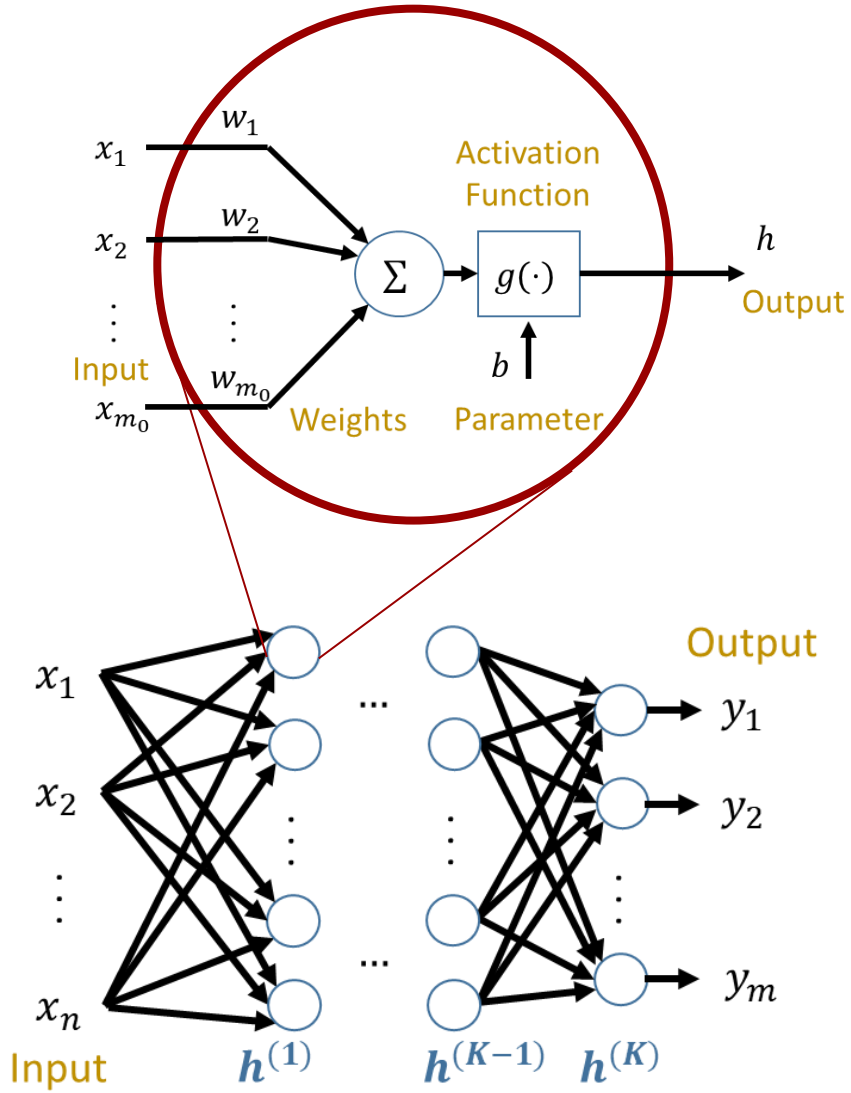
- If g is the identity map, we get a linear regression model
- If g is the soft-max function, we get soft-max regression



MLP Capacity

- Consider a 3-layer neural network used for a 3-class classification problem with an input $x \in \mathbb{R}^2$. Consider 16 neurons per hidden layer. How many weights and biases do you have per layer?

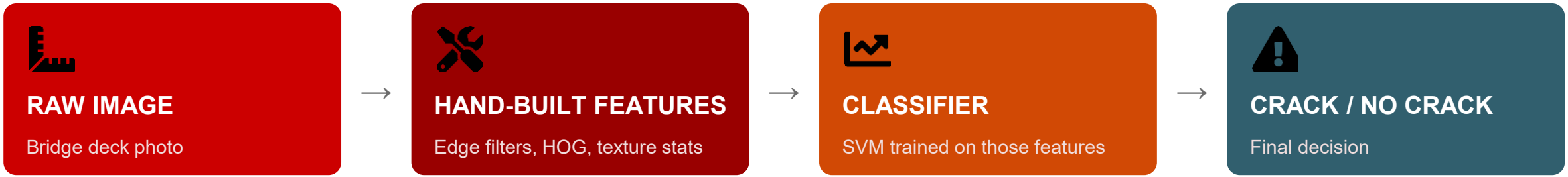
Feature Extraction	$h^{(0)} = x$
	$h^{(1)} = g^{(1)}\left(W^{(1)}h^{(0)} + b^{(1)}\right)$
	$h^{(2)} = g^{(2)}\left(W^{(2)}h^{(1)} + b^{(2)}\right)$
Classification Head	$h^{(3)} = g^{(3)}\left(W^{(3)}h^{(2)} + b^{(3)}\right)$
	$\hat{y} = h^{(3)}$



NC STATE Engineering

When Hand-Built Features Hit Their Limit

Bridge-deck crack detection — a classical computer vision pipeline



This pipeline works well on the bridge, lighting, and concrete texture it was tuned on. A different bridge, a different camera angle, or a different concrete mix often means starting the feature engineering over.

The bottleneck isn't the classifier — it's everything that has to happen by hand before the classifier ever sees the data.

NC STATE Engineering

Where Classical ML Hits a Wall

Four recurring limits, four different engineering examples

Manual Feature Engineering

Bearing fault detection needs hand-crafted FFT peaks and statistical features re-built for every new machine.

Struggles with Raw, High-Dim Data

Raw acoustic-emission waveforms for pipeline leak detection are too high-dimensional to feed in directly.

Limited Hierarchical Structure

Assessing earthquake damage from drone imagery needs edges → shapes → components → damage — layers classical models don’t build.

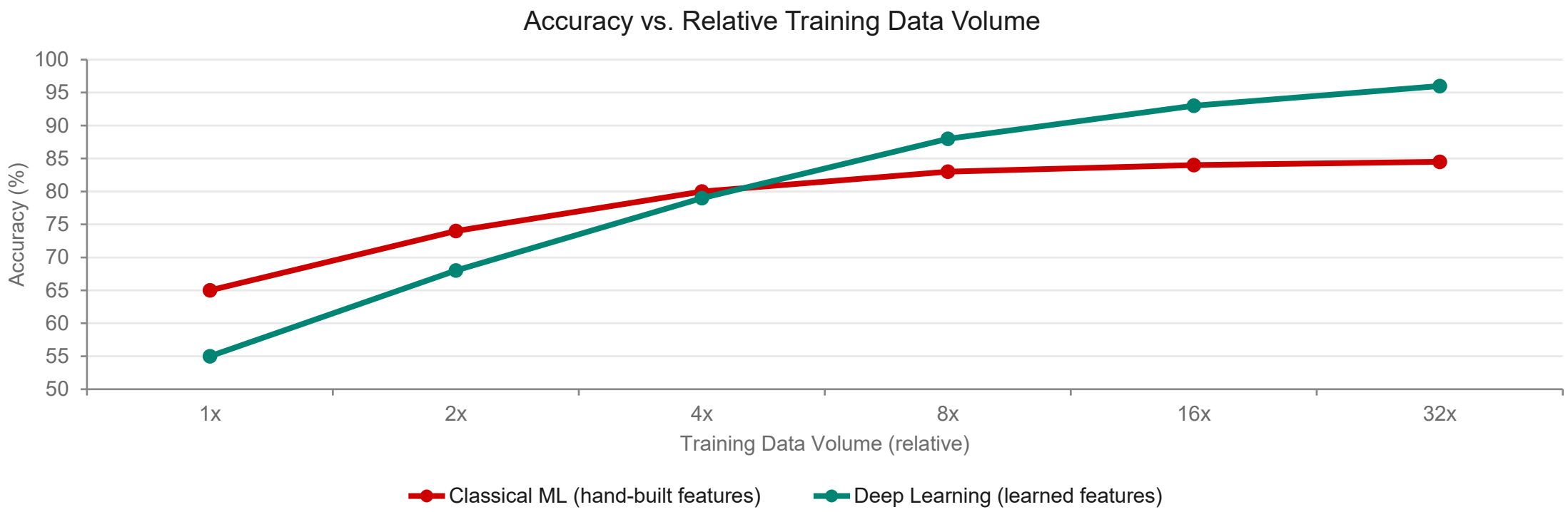
Multimodal Fusion Is Awkward

Fusing camera + radar + LiDAR for autonomous vehicles means hand-engineering features and fusion rules per modality.

NC STATE Engineering

The Performance Plateau

More training data doesn't rescue a classical pipeline forever



Classical accuracy tends to flatten well before you run out of data to give it — deep learning keeps improving instead.

NC STATE Engineering

Why Neural Networks & Deep Learning Help

The same four limits, answered — with four new examples

Automatic Feature Learning

A CNN learns crack patterns directly from raw pavement images — no hand-built edge or texture filters required.

Built for Hierarchical Data

Recognizing structural components in drone imagery benefits from layered edge → shape → component representations, learned automatically.

Multimodal Fusion Becomes Learnable

Your CRISI rail-crossing work fuses RGB, thermal, multispectral, and LiDAR via attention — learned, not hand-built.

Keeps Scaling, and Transfers

A model pretrained on a large dataset fine-tunes for weld-defect detection with a few hundred labeled images — and keeps improving with more data.

NC STATE Engineering

What Neural Networks Cost You

The honest counterweight — four costs, four examples

Data-Hungry

Rare failure modes — like bridge collapse events — are scarce, making it hard to gather enough labeled examples.

Compute-Intensive

Training a vision transformer for multimodal rail inspection needs multi-GPU clusters and careful CUDA management.

Black-Box / Hard to Interpret

A model flagging a structural anomaly with no explainable justification is a real problem for engineering sign-off.

Overfitting & Fragility

Small, even adversarial, perturbations — a sticker on a stop sign, sensor noise — can flip a prediction unexpectedly.

Deployment matters too: model size and latency are real constraints outside the datacenter — e.g., real-time inspection on a Jetson-class edge device.

NC STATE

Neither approach is strictly better.

Classical ML wins when data is scarce, interpretability is required, or the problem is simple.

Deep learning earns its cost when the data is complex, high-dimensional, or multimodal.

NEXT: HOW DO WE BUILD NEURAL NETS?